GASGRID

Smart LPG Booking & Distribution Management

Expanded technical report • OS + DBMS concepts mapped to working modules

Layer	Proposed technology	Role
Frontend	HTML, CSS, JavaScript	Customer portal, admin console, visual dashboards
Backend	Python Flask REST API	Validation, business rules, authorization, transactions
OS simulation engine	C	Scheduling, synchronization, deadlock, memory and file/disk simulations
Database	Oracle Database (or MySQL if mandated)	Persistent relational data, concurrency control, reports
Visualization	JavaScript charts / tables	Gantt charts, queues, resource and memory views
Version control	Git + GitHub	Source history, team collaboration, documentation and tests

Revised scope: scalable by design, with core features and optional OS labs

1. Project Overview

GasGrid is a web-based LPG booking and distribution management system for customers, distributors, technicians and administrators. It supports refill bookings, family connections, delivery tracking, mechanic requests, rewards, inventory, vehicle/worker allocation and reporting.

The project's differentiator is a dedicated OS Control Centre. It models bookings and service jobs as simulated processes and demonstrates how scheduling, synchronization, limited resources, deadlocks, memory and disk/file-system algorithms work. The DBMS stores the application's persistent facts and uses constraints and transactions to protect consistency. OS demonstrations run in a controlled simulation; they do not replace the real operating system scheduler.

2. Goals and Scope

- Deliver role-based customer and administration interfaces with clear navigation.
- Maintain reliable records for users, LPG connections, bookings, cylinders, dispatches, technicians, rewards and audit history.
- Map real business scenarios to OS concepts and expose results in visual form.
- Demonstrate a broad set of syllabus topics through interactive test cases, not merely definitions.
- Use DBMS normalization, constraints, joins, transactions, locking, views, indexes and optional PL/SQL routines.
- Keep modules independent enough that advanced simulations can be added without rewriting booking and inventory flows.

3. Users and Frontend Modules

Screen / module	Main functions	OS / DBMS connection
Login & registration	Authentication, role-based redirect, profile	Password hashing; Users/Role data; authorization
Customer dashboard	Connection summary, active booking, notices	Joins/views; status aggregation
Refill booking	Create/cancel booking, history, status	Process creation/state; transaction; booking indexes
Family connections	Link permitted members/connections	FK constraints; authorization; concurrent booking control
Delivery tracking	Status timeline, estimated stage, proof/log	Delivery state transitions; audit log
Mechanic services	Request technician, slot and priority	Priority scheduling; technician availability locking
Rewards	Balance, earn/redeem points, ledger	Atomic transaction; lost-update prevention
Inventory & warehouse	Cylinder stock, warehouses, vehicles, workers	Counting semaphore simulation; row locking; stock reports
Dispatch planner	Assign jobs to vehicles and workers	Resource allocation; scheduling; route/assignment constraints
OS Control Centre	Processes, Gantt charts, queues, memory, deadlock and disk demos	C engine results; saved simulation runs
Reports & audit	Bookings, low stock, pending delivery, service history	SQL joins, views, aggregates, indexes, audit trail
Admin settings	Manage service zones, capacities and simulation settings	RBAC; validation; configuration tables

UI direction: warm yellow/amber for primary actions, deep charcoal/navy sidebar, off-white content panels. Use green for completed, amber for waiting, and red only for errors/critical alerts. Keep customer navigation separate from admin and OS simulation navigation.

4. Architecture and Data Flow

Component	Responsibility and boundary
Frontend	Collects inputs and renders data; client-side checks improve usability but are not security controls.
Flask API	Authenticates and authorizes requests, validates inputs, applies business rules, manages DB transactions and returns JSON.
C OS engine	Runs deterministic simulations from validated input: process scheduling, synchronization/resource allocation, Banker's algorithm, memory/page replacement and disk scheduling.
DBMS	Stores durable application records and simulation metadata; enforces keys, constraints, transactions and access privileges.
Integration contract	Flask invokes the compiled C program using structured input/output (e.g., JSON via stdin/stdout or a local process wrapper). Validate all

	input; never execute arbitrary user text as a command.

System-of-record rule: the DBMS is authoritative for actual bookings, stock and rewards. The C engine can simulate resources using a snapshot; only backend transactions may commit real inventory or reward changes. Save simulation inputs and results separately from live operational records.

5. Operating System Concepts: Application Mapping

OS concept	Where it is used / how to demonstrate it
Program vs process; PCB	Show a booking program as code/template and each submitted job as a process record containing PID, state, arrival, burst, priority and resource needs.
Process states and transitions	Visualize New → Ready → Running → Waiting/Blocked → Terminated; show why a job waits for stock, a technician or an event.
Process creation / termination	Create a simulated process when a job enters the engine; cancellation/completion terminates it and releases simulation allocations.
Suspension / resumption	Pause and resume a simulation run or hold a job until a stock-restock event.
User vs kernel threads / multithreading models	OS Control Centre teaching panel comparing many-to-one, one-to-one and many-to-many models; not required to implement a real threading runtime.
fork/exec and signals	Optional C lab demonstrating child process creation, execution and controlled signal handling in an isolated local test mode.
CPU–I/O burst cycle	Model jobs as CPU/simulation work and waiting phases such as resource wait or dispatch delay.
FCFS, SJF, SRTF, Priority, RR	Run the same sample jobs through multiple schedulers and compare Gantt chart, waiting, turnaround and response time.
Multilevel queue / feedback queue	Separate job classes (e.g., urgent service, standard refill, reporting) and demonstrate fixed queues or dynamic priority changes.
Scheduling criteria	Calculate CPU utilization (within simulation), throughput, waiting time, turnaround time and response time.
SMP, load balancing, processor affinity	Optional visual lab showing jobs assigned across simulated CPU cores and comparing load distribution/affinity; clearly marked as a model.
IPC: shared memory vs message passing	Explain and optionally simulate competing processes sharing a stock counter versus sending booking messages to a dispatcher queue.
Race condition / critical section	Run two concurrent simulated booking attempts against limited stock; show an unsafe interleaving versus protected execution.
Mutex, binary and counting semaphores	Protect shared stock/reward updates; counting semaphore models a fixed number of cylinders, vehicles or technician slots.
Peterson's solution	Two-process educational demo with a shared critical-section counter; separate from production inventory control.
TestAndSet / CompareAndSwap / atomic variables	Show atomic lock/counter examples in the OS lab; use safe, platform-appropriate C implementation if implemented.
Producer–Consumer / bounded buffer	Booking requests are producers; dispatch worker is consumer; queue capacity and full/empty states are visible.
Readers–Writers	Many users/admins can read stock reports while a controlled update requires exclusive write access in the simulation.
Dining Philosophers	Generic resource-sharing simulation demonstrating circular wait and prevention; not tied to actual customer data.
Deadlock conditions and RAG	Visualize mutual exclusion, hold-and-wait, no preemption and circular wait using vehicles, cylinders and technician slots.
Deadlock prevention / avoidance / detection / recovery	Toggle resource-allocation policies; compare safe ordering, detection and simulated recovery/rollback.
Banker's Algorithm	Show Available, Max, Allocation, Need; run safety and resource-request algorithms with safe/unsafe sample states.
Memory allocation	First-Fit, Best-Fit and Worst-Fit allocate simulated memory blocks to process jobs; show internal/external fragmentation.
Paging / page tables	Optional page mapping panel translates logical page numbers to frame numbers and demonstrates page-table lookup.
Page replacement	FIFO, Optimal, LRU, Second-Chance/Clock, LFU/MFU on a reference string; show page faults and Belady's anomaly where applicable.
Working set / thrashing / PFF	Optional graph of page-fault rate and working-set size as memory pressure increases.
File concepts and directory structures	Model simulation-run files/logs, metadata, directory tree and access methods in an educational file-system panel.
File allocation and free-space management	Visualize contiguous, linked and indexed allocation plus bitmap/free-list tracking on a mock disk.
Disk scheduling	FCFS, SSTF, SCAN, C-SCAN, LOOK and C-LOOK compare head movement for a request queue.
Protection and access matrix	Demonstrate least privilege and a role/object permission matrix for Customer, Distributor, Technician and Admin.
Swapping / virtual memory / NUMA	Optional advanced comparison panels; do not imply that the website directly controls host OS memory.

6. DBMS Concepts: Application Mapping

DBMS concept	Use in GasGrid

ER modelling and cardinality	Define one-to-many relationships such as User→Bookings, Warehouse→Stock records and Booking→Delivery; document optional relationships.
Normalization (1NF–3NF)	Separate users, connections, bookings, inventory, deliveries, technicians and rewards rather than duplicating repeated data.
Primary, foreign, candidate and unique keys	Identify records reliably and enforce relationships; unique email/connection number/vehicle registration as appropriate.
Domain, NOT NULL, CHECK constraints	Restrict statuses, positive quantities, valid points and required fields at database level.
Joins and subqueries	Customer history, family connection bookings, pending deliveries, low stock, technicians with open jobs.
Aggregate functions and GROUP BY/HAVING	Daily bookings, delivery completion rate, stock by warehouse, three-or-more bookings per zone, reward summaries.
Set operations	UNION/UNION ALL for consolidated job lists; INTERSECT/MINUS or DBMS-equivalent where supported for report comparisons.
Views	Provide role-appropriate dashboard/reporting datasets without repeating complex SQL in every endpoint.
Indexes and query plans	Index frequently filtered keys/status/date columns; measure query plans before adding unnecessary indexes.
Transactions and ACID	Booking/dispatch/reward redemption updates all related rows together; rollback on failed validation or error.
Concurrency control / isolation / locks	Prevent overselling stock and double redemption using row locks or conditional atomic updates; choose isolation intentionally.
COMMIT, ROLLBACK, SAVEPOINT	Demonstrate successful dispatch, failed dispatch rollback, and a recoverable multi-step admin operation.
PL/SQL procedures and functions (Oracle)	Optional procedures for reports or controlled inventory operations where they add clarity.
Triggers	Optional audit trigger; avoid duplicating core business rules across triggers and Flask.
Sequences / identity columns	Generate IDs for bookings, deliveries, service requests and simulation runs.
Cursors (if required by syllabus)	Use in a focused PL/SQL report/demo rather than for ordinary queries that a set-based query can handle.
Roles, privileges and least privilege	Backend database account receives only required privileges; separate admin application role from database superuser.
Audit logs	Record sensitive state changes with actor, action, entity, timestamp and result.
Backup, recovery and data integrity	Document backup/restore approach and test restoring a development database.
Deadlocks at database level	Show how competing transactions can wait on locks; keep transactions short and use consistent update order.
Relational reporting	Build dashboard metrics with joins, aggregates and date filters; paginate large history lists.

7. Proposed Core Database Schema

Table	Illustrative key fields
Users	user_id PK, name, email UNIQUE, password_hash, role, phone, created_at
Roles / Permissions (optional)	role_id PK, role_name; permission_id PK, resource, action; role-permission mapping
LPG_Connections	connection_id PK, user_id FK, connection_number UNIQUE, address, status
Family_Members	family_member_id PK, connection_id FK, member_name, relationship, access_level
Bookings	booking_id PK, connection_id FK, booking_date, status, priority, cylinder_type, created_at
Warehouses	warehouse_id PK, name, service_zone, location
Inventory_Batches	batch_id PK, warehouse_id FK, cylinder_type, available_qty, reserved_qty, updated_at
Cylinders (if unit-level tracking is needed)	cylinder_id PK, batch_id FK, serial_no UNIQUE, cylinder_status
Vehicles	vehicle_id PK, registration_no UNIQUE, capacity, status
Workers	worker_id PK, name, role, availability_status
Deliveries	delivery_id PK, booking_id FK, vehicle_id FK, worker_id FK, dispatch_time, status
Technicians	technician_id PK, worker_id FK, service_zone, skill, availability_status
Service_Requests	request_id PK, user_id FK, technician_id FK nullable, slot, priority, status
Rewards_Ledger	ledger_id PK, user_id FK, points_change, reason, transaction_time
OS_Simulation_Runs	run_id PK, algorithm_or_lab, created_at, input_json, result_json, created_by FK
OS_Simulation_Processes	sim_process_id PK, run_id FK, source_request_id nullable, arrival, burst, priority, final_state
Audit_Logs	log_id PK, actor_user_id nullable, action, entity_type, entity_id, created_at
Service_Zones / Restock_Events (optional)	zone_id PK, zone_name; event_id PK, warehouse_id FK, quantity,

	event_time, reason

Choose one inventory model deliberately: quantity-based batches are simpler; individual cylinder rows allow unit-level tracking. Avoid maintaining both independently unless reconciliation rules are defined. Never store plain-text passwords. Keep OS simulation state distinct from real booking state.

8. Example Workflow: Booking to Delivery

1. A customer signs in and submits a refill request. The frontend sends the request to Flask.
2. Flask checks authorization, validates the connection and creates the booking inside a database transaction.
3. The backend sends a sanitized job description to the C engine. The engine assigns a simulated PID and process state.
4. The scheduler selects the job. The resource simulator checks cylinder/vehicle/worker availability; unavailable resources place the process in Waiting.
5. The OS Control Centre displays the process queue, Gantt chart, metrics and resource availability.
6. When dispatch is authorized, Flask rechecks live inventory and commits reservation, delivery assignment and status updates atomically.
7. If any constraint or stock check fails, the transaction rolls back and the user sees a clear error.
8. Delivery status changes are recorded with audit events; the simulated process is completed and the run result is saved.

Example: three refill requests arrive while only two simulated cylinders are available. Two jobs receive simulated allocations and one waits. When a restock event adds stock, the waiting job can be reconsidered. Actual dispatch independently checks and reserves real database stock.

9. API Outline

Route (proposed)	Purpose
POST /api/auth/login	Authenticate and establish session/token
GET /api/dashboard	Role-specific dashboard summary
POST /api/bookings; GET /api/bookings	Create and list authorized bookings
PATCH /api/bookings/{id}/cancel	Cancel an eligible booking
GET /api/inventory	Authorized stock view
POST /api/inventory/restock	Record validated restock event
POST /api/deliveries/{id}/dispatch	Transactional stock check and dispatch
POST /api/services/requests	Create mechanic request
POST /api/rewards/redeem	Atomic points validation and redemption
POST /api/os/simulations	Run a validated OS simulation
GET /api/os/simulations/{id}	Return queue, Gantt data, metrics and results
GET /api/reports/stock; GET /api/reports/deliveries	Authorized reporting endpoints

These routes are design proposals, not implemented endpoints. Apply authentication, authorization, input validation, error handling and pagination consistently.

10. OS Control Centre: Suggested Tabs

Tab	Visuals and interactions
Process & Scheduling	Process table, state badges, algorithm selector, Gantt chart, waiting/turnaround/response metrics
Synchronization Lab	Race-condition timeline, lock/semaphore state, producer-consumer bounded buffer, readers-writers demo
Resource & Deadlock Lab	Available/Max/Allocation/Need matrices, resource-allocation graph, safety sequence and recovery outcome
Memory Lab	Memory blocks, First/Best/Worst Fit, page table, page-fault counts, FIFO/LRU/Clock comparison
Disk & File-System Lab	Disk request queue/head movement, allocation method visualizer, free-space bitmap and directory tree
Protection Lab	Role-permission matrix, least-privilege examples and access-denied test cases
Run History	Saved simulation inputs/results, algorithm, timestamp and repeat-run option

11. GitHub Repository Structure

```
GasGrid/
├── README.md
├── .gitignore
├── .env.example
├── docs/
│   ├── project-report.docx
│   ├── architecture.md
│   ├── er-diagram.png
│   ├── api-contract.md
│   └── test-plan.md
├── frontend/
│   ├── pages/
│   ├── css/
│   ├── js/
│   └── assets/
├── backend/
│   ├── app.py
│   ├── routes/
│   ├── services/
│   ├── auth/
│   ├── db/
│   └── requirements.txt
├── os_engine/
│   ├── src/
│   │   ├── main.c
│   │   ├── process.c
│   │   ├── scheduling.c
│   │   ├── synchronization.c
│   │   ├── banker.c
│   │   ├── memory.c
│   │   ├── page_replacement.c
│   │   ├── disk_scheduling.c
│   │   └── filesystem_demo.c
│   ├── include/
│   ├── tests/
│   └── Makefile
├── database/
│   ├── schema.sql
│   ├── constraints_indexes.sql
│   ├── seed_data.sql
│   ├── queries.sql
│   ├── views.sql
│   └── procedures_triggers.sql
└── tests/
    ├── api/
    ├── database/
    └── scenarios/
```

Commit source, SQL, documentation, tests and synthetic seed data. Exclude passwords, API secrets, real customer data, database dumps, compiled binaries, virtual environments and local configuration. Keep .env ignored and publish only a safe .env.example.

12. Four-Member Team Plan

Member focus	DBMS ownership	OS ownership
1. Accounts & family	Users, roles, connections, family constraints and joins	Process creation, PCB fields, states, suspension/termination
2. Inventory & resources	Warehouses, inventory transactions, constraints, locks	Critical sections, mutex/semaphore, producer-consumer, resource allocation
3. Booking & delivery	Bookings, deliveries, services, SQL reports	FCFS/SJF/SRTF/Priority/RR and scheduling metrics
4. Integration & analytics	Rewards, audit, views/indexes, transaction testing	Banker's algorithm, deadlock demo, memory/disk labs and API integration

All members should be able to explain the end-to-end flow, ER relationships, transaction boundaries, scheduler calculations and why the C engine is a simulation.

13. Implementation Roadmap

Phase	Deliverable
1. Requirements & ER design	Finalize roles, workflows, inventory model, ER diagram and database choice.
2. DBMS foundation	Schema, constraints, seed data, joins, views, indexes, transaction and rollback tests.
3. Core application	Login, customer dashboard, booking, history, inventory, mechanic and delivery pages.
4. OS engine foundation	Process structure/state transitions; FCFS and SJF or RR; validated input/output format.
5. Integration	Flask API connects to DBMS and C engine; show persisted simulation runs and Gantt chart.
6. Synchronization & deadlock	Shared-stock race scenario, mutex/semaphore demo, bounded buffer and Banker's Algorithm.
7. Advanced OS labs	Memory allocation, page replacement, disk scheduling, file allocation and protection matrix.
8. Quality & presentation	Concurrency tests, failure/rollback scenarios, access checks, screenshots, README and viva preparation.

14. Testing and Demonstration Checklist

- Customer cannot access admin-only inventory, reports or simulation controls.
- Duplicate/invalid connection requests are rejected and clear validation errors are returned.
- Two concurrent dispatch attempts cannot reduce stock below zero.
- Failed dispatch and reward redemption roll back all related database updates.
- Scheduling output matches manually calculated test cases for known arrival/burst times.
- Semaphore and bounded-buffer scenarios correctly handle full/empty states.
- Banker's safety and request algorithms correctly label test states safe or unsafe.
- Memory allocation and page replacement results match hand-worked examples.
- Disk scheduling reports correct request order and total head movement.
- Database credentials use least privilege; secrets and real personal data are absent from GitHub.
- Simulation records are clearly separated from actual booking and inventory records.

15. Final Recommendation

Because there is no strict limit on concepts or scalability, GasGrid can include a wider OS learning suite while keeping its real business workflow coherent. Make booking, inventory, transactions and role-based access the application core. Add scheduling, process states, synchronization, IPC and Banker's Algorithm as the primary OS integration. Then add memory/page replacement, disk scheduling, file-system allocation, protection/access matrix, multilevel queues and simulated multicore load balancing as separate Control Centre labs. Advanced concepts such as scheduler activations, NUMA and detailed signal/thread behavior may be included as clearly labelled educational demos if time permits.

For academic clarity, distinguish three categories in the presentation: (1) implemented business functionality, (2) integrated OS simulations driven by GasGrid scenarios, and (3) standalone educational labs using sample inputs. This makes the project broad without claiming that a web application directly controls the host operating system.